Project Tasks

[X] 1. Initialize the Project

- Set up the project directory and create necessary files (`README.md`, `.gitignore`, `requirements.txt`).
- Initialize a Git repository and push it to GitHub.

[X] 2. Install Dependencies

- Install required Python packages: `pyspark`, `kafka-python`, `requests`, `beautifulsoup4`, `pyspark-cassandra`, etc.
- Set up a virtual environment or use `conda` for managing dependencies.

[X] 3. Google Maps API Setup

- Set up a Google Maps API key for collecting user location data and nearby places.
- Write a script to fetch user locations and nearby places based on search queries.

[X] 4. Weather API Setup

- Set up API access (e.g., OpenWeatherMap or WeatherStack).
- Write a script to fetch weather data (temperature, conditions) for given locations.

[ ] 5. Reviews Collection

- Choose social media platforms or APIs (e.g., Twitter, Reddit, or scraping using `BeautifulSoup`).
- Write scripts to scrape reviews for locations.

[ ] 6. Preliminary Data Collection

- Run the scripts and collect data for user locations, weather, and reviews.
- Store the data in `data/raw/`.

[ ] 7. Data Preprocessing Functions

- Write functions to format and clean the collected data.
- Implement functions to convert data into a consistent structure (e.g., UUID for locations, user IDs).

[ ] 8. Kafka Setup

- Set up Kafka locally or use a Dockerized version.

- Write a Kafka producer to stream preprocessed data into Kafka topics (user preferences, locations, reviews, and weather).

[ ] 9. Stream Data to Kafka

- Use `kafka-python` to stream preprocessed data to Kafka topics.
- Ensure data integrity and correctness by consuming data from Kafka using a Kafka consumer script.
- Store the output in `data/kafka_output/`.

[ ] 10. Spark Setup

- Set up Apache Spark on your machine or in a Docker container.
- Install `pyspark` and `pyspark-cassandra` for integration with Kafka and Cassandra.

[ ] 11. Spark Streaming for Kafka

- Write a Spark Streaming job to read data from Kafka.
- Process the stream to match user location data with weather data.
- Implement a function to find nearby locations within a 1km radius of each user.

[ ] 12. Test Spark Streaming

- Test the Spark job by reading and processing a small batch of data from Kafka.

[ ] 13. Set up Cassandra

- Install and configure Cassandra on your machine or use a Dockerized version.
- Set up keyspaces and tables in Cassandra for storing processed data.

[ ] 14. Integrate Spark with Cassandra

- Write a script to insert processed data from Spark into Cassandra tables.
- Use `pyspark-cassandra` to connect Spark to Cassandra and store processed data.

[ ] 15. Test Cassandra Integration

- Check if the processed data is correctly stored in Cassandra after processing by Spark.
- Verify data integrity and correctness in Cassandra.

[ ] 16. Sentiment Analysis Model

- Implement sentiment analysis using Spark MLlib on user reviews.
- Prepare a dataset of reviews and convert them into feature vectors (e.g., using `CountVectorizer` or `TF-IDF`).
- Train a simple sentiment analysis model (e.g., Logistic Regression).

[ ] 17. Apply Sentiment Analysis

- Use the trained model to score nearby locations based on sentiment.
- Add sentiment scores to the locations' data (in addition to weather and review scores).

[ ] 18. Test Sentiment Analysis

- Run the sentiment analysis model on a sample batch of review data to test accuracy and performance.

[ ] 19. Collaborative Filtering

- Implement collaborative filtering to recommend nearby locations based on user preferences.
- Use user interactions (e.g., locations visited) and identify similar users to generate recommendations.

[ ] 20. Content-Based Filtering

- Implement content-based filtering to recommend locations with specific attributes (e.g., high review score, good weather).
- Combine it with collaborative filtering in a hybrid model.

[ ] 21. Hybrid Recommendation System

- Combine both collaborative and content-based recommendations to generate a final list of recommended locations.

[ ] 22. Generate Final Output

- Generate a JSON output containing the final recommendations for each user, including reasons for recommendations (e.g., sentiment score, weather conditions). Example output: "'json [{ "user_id": "u101", "recommended_locations": [{ "location_id": "l201", "score": 4.9, "reason": "Positive sentiment and clear weather." }, { "location_id": "l203", "score": 4.85, "reason": "Positive sentiment and proximity." }] }]